



Weather Trends Project

Introduction

Hi 🙋

In my first project on **Udacity DAND**, I analyzed the temperature around the global with the city I live by extracting the data from the database.

Tools

- SQL (PostgreSQL🐘) is to data extracting and copying.
- 🐍 Python and its most popular packages like NumPy, Pandas, Matplotlib and FastAPI
- ✓ Angular Front-end Framework
- Jupyter Notebook 😊

Extracting Data Process

For this step, I had to use some SQL queries to know more about datasets. For getting all cities in the country, I wrote this:

```
SELECT *
FROM
  city_list
WHERE country="Uzbekistan"
-- out: Tashkent
```

The output of the query which is given above returned one city with the name of our country's capital. This is Tashkent!

In this project, for extracting data that is related to my city, I used SQL join query to extract it with global data. I wrote like this query (you can find it in my repository):

```
SELECT
  c.year AS c_year,
```

```

c.city,
c.country,
c.avg_temp AS c_temp,
g.year AS g_year,
g.avg_temp AS g_temp
FROM city_data AS c
JOIN global_data g ON c.year = g.year
WHERE c.city LIKE 'Tashkent'

```

Result:

Accessing Data With SQL

SEND FEEDBACK

Input

HISTORY MENU

```

1 SELECT
2   c.year AS c_year,
3   c.city,
4   c.country,
5   c.avg_temp AS c_temp,
6   g.year AS g_year,
7   g.avg_temp AS g_temp
8 FROM city_data AS c
9 JOIN global_data g ON c.year = g.year
10 WHERE c.city LIKE 'Tashkent'

```

Success!

EVALUATE

Output 182 results

Download CSV

c_year	city	country	c_temp	g_year	g_temp
1832	Tashkent	Uzbekistan	9.48	1832	7.45
1833	Tashkent	Uzbekistan	9.93	1833	8.01
1834	Tashkent	Uzbekistan	9.95	1834	8.15
1835	Tashkent	Uzbekistan	9.15	1835	7.39
1836	Tashkent	Uzbekistan	9.65	1836	7.70
1837	Tashkent	Uzbekistan	9.32	1837	7.38
1838	Tashkent	Uzbekistan	9.02	1838	7.51
1839	Tashkent	Uzbekistan	9.05	1839	7.63
1840	Tashkent	Uzbekistan	9.63	1840	7.80
1841	Tashkent	Uzbekistan	9.13	1841	7.69
1842	Tashkent	Uzbekistan	10.10	1842	8.02

Menu Shrink

NEXT

After that, I have downloaded this and all data from my classroom's workspace 😊
 Then, I could reach to create functionality that allows you to build plots of any city that are listed in the city list **CSV** file!

Opening up the CSV and ...

I haven't used tools like excel or google spreadsheets. I prefer to use Python in my projects. For this process, I think the `pandas` is very important thing. It helped me to work with CSV files. I created `utils` module, for creating functionality which helps to build the plot of any city. With `os.path` and `pandas`' `read_csv()` we can easily read CSV files!

```

# web/utils/__init__.py
...
def get_path(self, file_name: str):

```

```

        return path.abspath(f'data/{file_name}.csv') # for getting absolute path of file

def init(self) -> tuple:
    return pd.read_csv(self.get_path('city_data')), \
           pd.read_csv(self.get_path('global_data')) # it returns two data frames
...

```

So necessary datasets are collected. Then, I have created method for getting data by given city name:

```

def getDataByCity(self, city: str) -> list:
    # By using conditions with data frames,
    # we can get data by city name

    lc_data = self.city_data.loc[
        self.city_data['city'] == city
    ]

    # Gets all data about years,
    # And define beginning and ending this year's period.

    years = lc_data.year
    beginning, ending = min(years), max(years)

    # Filtering global data by beginning and
    # ending of the period which equally with local data's period

    gl_data = self.global_data[
        (beginning <= self.global_data.year) &
        (self.global_data.year <= ending)
    ]
    return [lc_data, gl_data]

```



Creating Line Chart

I have added a new method with the name `make_plot` on my `WeatherTrends` object. This method helps us to create line charts.

For calculating "moving average", I used pandas' methods. The combination of methods which are `rolling()` and `mean()`, calculates this average easily for you 😊 `rolling()` method of pandas provides us the feature of rolling window calculations. And, for next step we should use aggregation like a `mean()` which allows us to calculate the rolling mean.

```

def make_plot(self,city_name:str):
    # get city name related data
    data = self.getDataByCity(city_name)
    local_data, global_data = data[0], data[1]

    # setting the first line-chart params
    x_global = global_data['year']
    y_global = global_data['avg_tmp'].rolling(10).mean()

    # setting the first line-chart params
    x_local = local_data['year']
    y_local = local_data['avg_tmp'].rolling(10).mean()

    # create new figure
    plt.figure(figsize=(30,30),dpi=80)

    # no comment ☺
    plt.plot(
        x_global, y_global,
        label='Global temperature'
    )

    plt.plot(
        x_local, y_local,
        label='{} temperature'.format(
            city_name.title()
        )
    )

    plt.title('Global and ' + city_name + ' average tempature over time')

    # configure plot labels which are being displayed on its axis
    plt.xlabel('Years',fontsize=20)
    plt.ylabel('Temperature in celcius',fontsize=20)

    # plot's legend position
    plt.legend(loc='upper right')

    return plt

```

Simple usage of this tool:

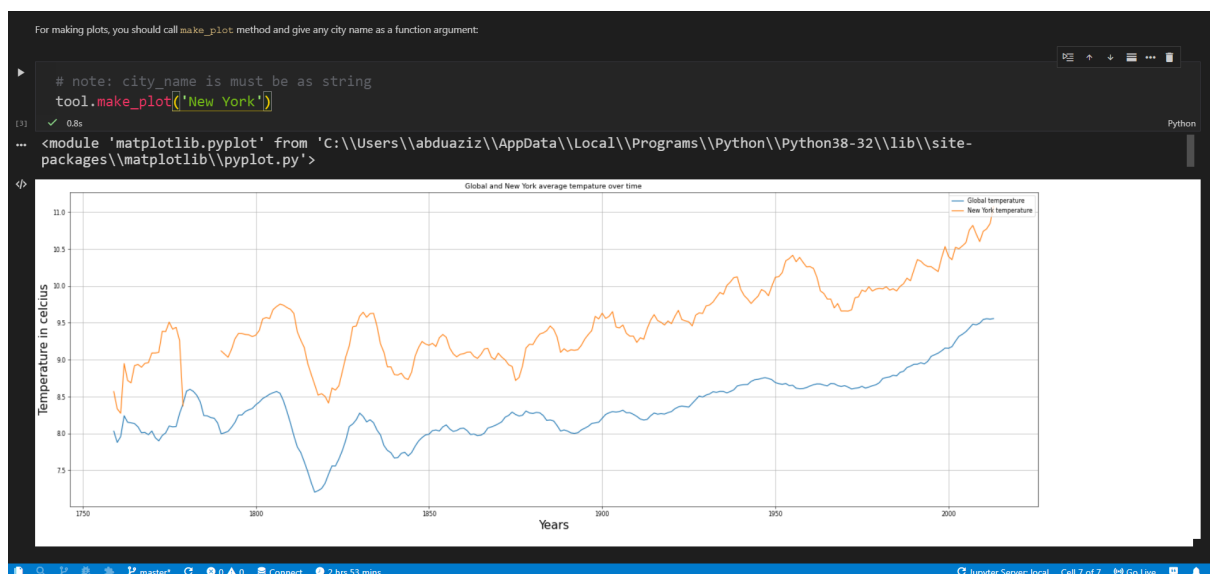
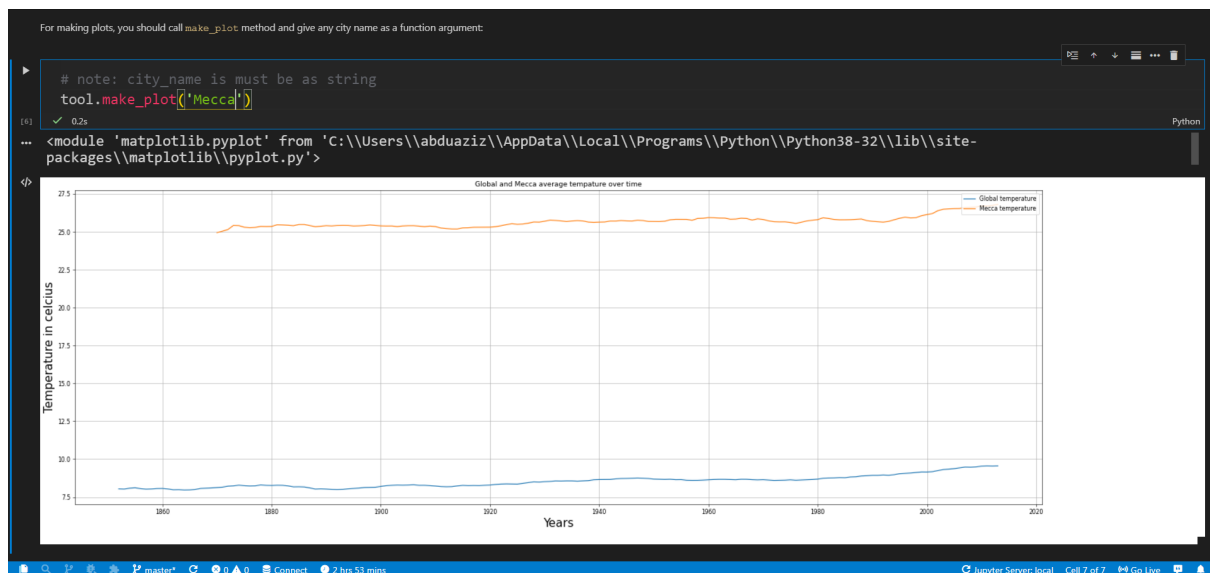
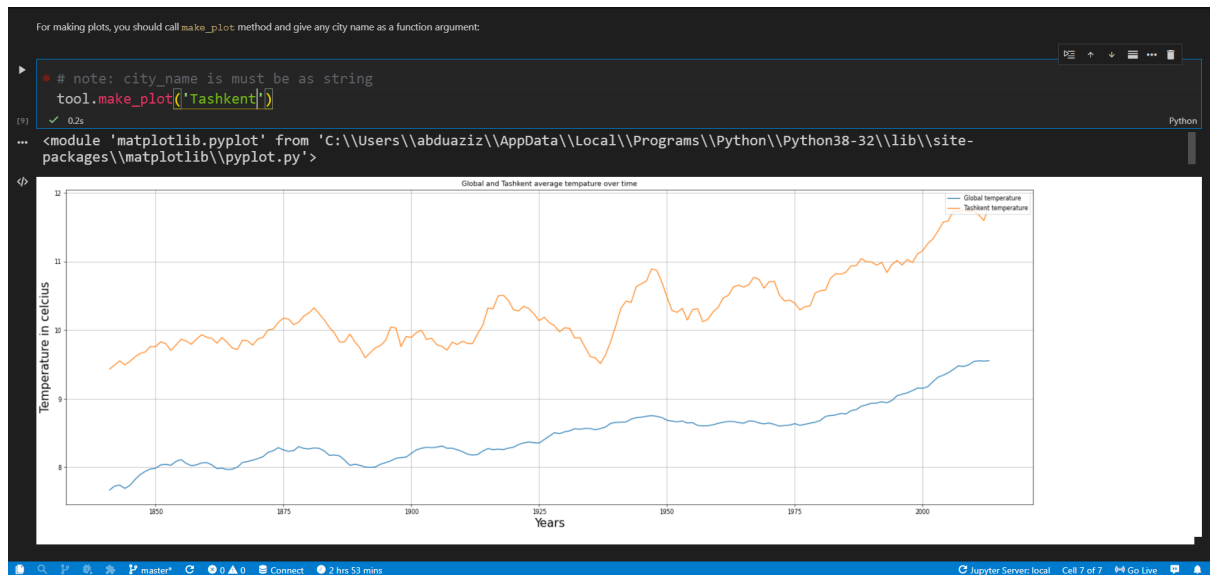
```

from web import WeatherTrends

tool = WeatherTrends()
tool.make_plot('Tashkent')

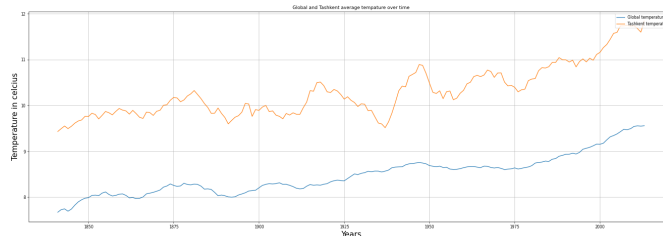
```

Results:



You can test any city name 😊

My observations



- Tashkent's temperature is absolutely hotter compared with the global temperature
- Between 1875 and 1950 had been more fluctuations on Tashkent's chart. There had been unpredictable temperatures, that is the temperature in this city significantly increased and decreased ...
- In the global temperature chart, we can see that the average temperature is had been marginally increasing year by year.
- My city's and global's chart has a very small difference in the temperature (*I could calculate this difference below*).
- I observed that, global temperature had increased from 7 °C to 9.61°C while my city's temperature had increased from 9.48°C to 13.66°C.
- Over the 180 years:
 - The average temperature is had increased by almost 3 °C in my city
 - My city's temperature had been higher than global temperature by approximately 2 °C (all time) I calculated this by using python:

```
import numpy as np
from web import WeatherTrends

tool = WeatherTrends()

data = tool.getDataByCity('Tashkent')

l, g = d[0], d[1]
```

```
l, g = np.array(l['avg_tmp']), np.array(g['avg_tmp'])

print( l.mean() - g.mean() ) # 1.82868... ~ 2
```

- I think world is getting hotter year by year due to changes in the climate.

Web Service

I have used my web development skills on this project. That is I made API that returns plots by given city name. I used FastAPI which is modern python web framework. I configured CORS settings in my API then I created two routes:

- `/` - basic route. Returns `{"message": "Welcome"}`
- `/api/v1/:city` - the route which is related to charts.

The second returns chart file's path in **JSON** format . I want to explain my code:

```
from os import listdir
...
api.mount(
    "/charts",
    StaticFiles(directory='charts'),
    name='charts'
)
...
async def CityDetail(city: str):
    # get all files using os.listdir
    charts:list = listdir('charts/')
    # we can define filename like this
    filename:str = city.title() + ".png"
    # if this file is exist
    # we do not have to create a chart again
    # we should just return this
    if filename in charts:
        return {
            "file": f"charts/{filename}"
        }
    # Or, filename is not exist
    # we should create line chart for this
    try:
        util.save(city.title())
    # if code raises any exceptions
    # we have to return http 404
    except:
        raise HTTPException(404)
    # if any errors are not returned
    # we can return response
    return {
        "file": f"charts/{filename}"
    }
}
```

We have to run this web service using this command:

```
$ uvicorn web:api --reload
# navigate http://127.0.0.1:8000/docs to see docs
```

GET / Welcome

Cities

GET /api/v1/{city} CityDetail

Returns all data which are related by city name

Parameters

Cancel

Name Description

city * required string (path) Tashkent

Execute Clear

Responses

Curl

```
curl -X GET -\
  "http://127.0.0.1:8000/api/v1/tashkent" \
  -H "accept: application/json"
```

Request URL

```
http://127.0.0.1:8000/api/v1/tashkent
```

Server response

Code Details

200

Response body

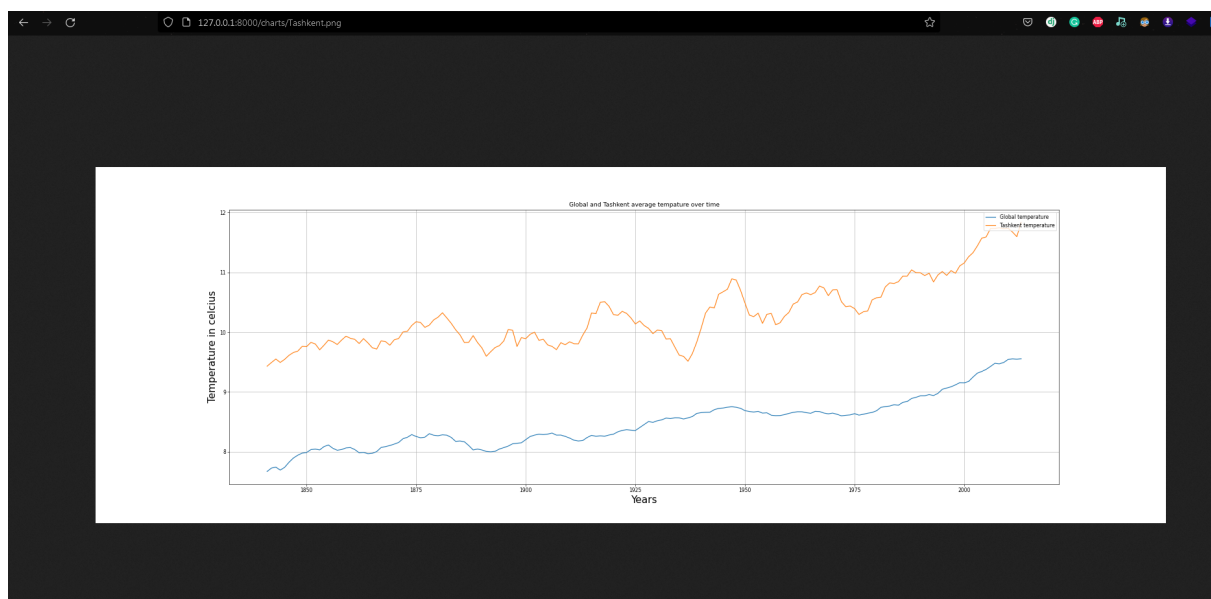
```
{
  "file": "charts/tashkent.png"
}
```

Response headers

```
content-length: 30
content-type: application/json
date: Mon, 05 Sep 2021 08:22:41 GMT
server: uvicorn
```

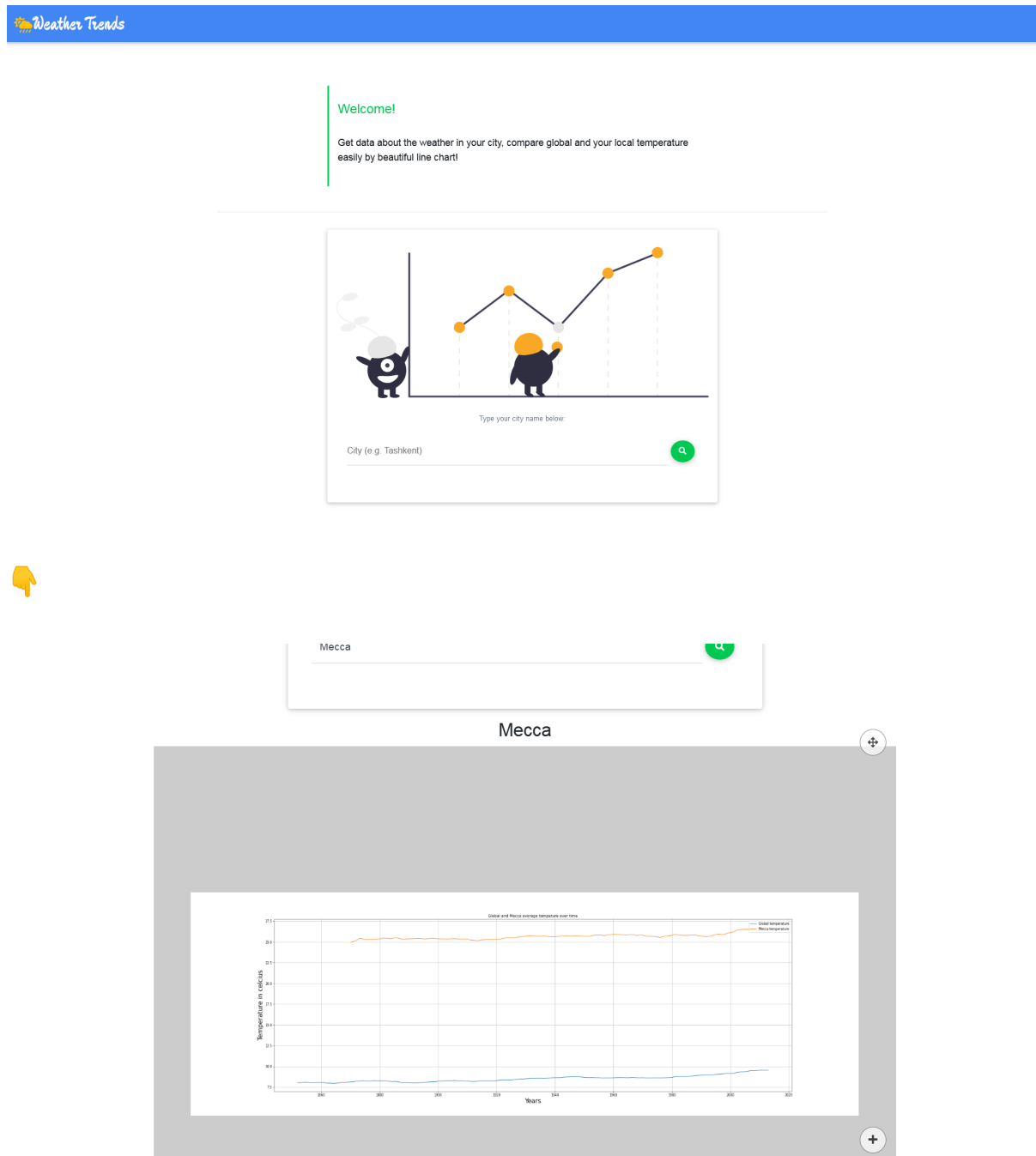
Responses

In the response body, we can see that it has a filename and its path. Now, we should go by this path to see the line chart:



I used Angular and Material Design Bootstrap Library for the front-end of this project. This front-end communicates with the back-end (API) to get data about charts. Users

have to type only city names to get a line chart.



I also used the image viewer library for allowing users to look at line charts in more detail.

Links:

- **Back-end** - <https://weather-trends-dand.herokuapp.com/docs>
- **Front-end** - <https://weather-trends-dand.netlify.app>

- **Project** - <https://github.com/AbduazizZiyodov/weather-trends>

Author: Abduaziz Ziyodov 😁